❶ Delegates the creation and termination of the ActorTestKit

❷ The test kit comes from ScalaTestWithActorTestKit.

❸ Sets the expectation

❹ Sets the code to trigger the logging

Note that there is no indication of which actor is logging what. The only assertion this test makes is that at the end, the text “it’s done” is logged in the `INFO` level. The expectation applies to the entire `ActorSystem`, unlike with the synchronous test kit, which applies only to an actor created with the `BehaviorTestKit`.

You may remember `deadLetters` from chapter 3: messages whose address no longer exists because their destination—the `ActorRef`—no longer exists. With the `LoggingTestKit`, you can easily catch the logging generated in `deadLetters` by using a regular expression that matches the logging event.

To re-create this scenario, all you need is a stopped actor, which you can create using the `Behaviors.stopped` factory:

```
val behavior: Behavior[String] = Behaviors.stopped
```

Sending a message to this stopped actor redirects it to `deadLetters`. You can test this with the `LoggingTestKit.empty` constructor by adding the desired `.withLogLevel` and the regular expression via `.withMessageRegex`.

Listing 4.6 Testing that some messages got lost

```
class AsyncLogSpec ...
  ...
  "log messages to dead letters" in {
```